TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT NO. 0

SOLID PRINCIPLES IN A SIMPLE TASK LIST

Student: Lucian-Adrian Gavril

Group: FAF-241

Coordinator: asist. univ., Daniel Istrati

Chișinău, 2026

TABLE OF CONTENTS

ABSTRACT 3
INTRODUCTION 4
1 PROJECT GOAL AND STRUCTURE 5
2 THREE PRINCIPLES IN THE SOURCE 6
2.1 Single Responsibility Principle 6
2.2 Open/Closed Principle 6
2.3 Dependency Inversion Principle 6
3 TRACE OF THE SAMPLE 7
4 REPRODUCTION AND REVIEW 8
CONCLUSIONS 9
REFERENCES 10
APPENDICES 11
APPENDIX A – Program source 11

ABSTRACT

The Laboratory Work No. 0 brief asks students to implement three SOLID letters in a simple project. This report traces a C# task-list example that demonstrates Single Responsibility, Open/Closed, and Dependency Inversion. It explains each principle using the types and methods in `Program.cs`, traces the seeded tasks through the source, and gives build commands and oral-review questions.

Keywords: SOLID, single responsibility, open/closed, dependency inversion, C#

INTRODUCTION

The assignment's stated goal is concise: implement three SOLID letters in a simple project [1]. It asks for an applied example rather than a list of definitions. The public brief page does not name a required trio, so this study example uses three principles that can be followed through one small flow: task data is provided by a source, selected by a filter, and printed by a report.

The course page groups the assignment under SOLID Principles and lists Robert C. Martin's *Design Principles and Design Patterns* (2000) among its references [2, 3]. This report uses the following selections:

a) Single Responsibility Principle (SRP): each class has one focused reason to change;
b) Open/Closed Principle (OCP): the report can accept a new filter implementation through an existing interface;
c) Dependency Inversion Principle (DIP): the report depends on source and filter abstractions, while the entry point supplies concrete implementations.

The report follows the source in Listing A.1. It distinguishes the code that exists from a possible extension, and its trace is derived from the seeded data rather than presented as a new measured run.

1 PROJECT GOAL AND STRUCTURE

The project models three tasks with an identifier, title, priority, and completion flag. `Program.Main` creates those values, selects the in-memory source and open-task filter, then constructs `TaskReport`. The report obtains the list, passes it through the filter, and prints the selected tasks.

Type	Role in the existing program
`TaskItem`	Holds one task's ID, title, priority, and done state (line 23).
`ITaskSource` / `InMemoryTaskSource`	Define and provide access to the task collection (lines 25–43).
`ITaskFilter` / `OpenTaskFilter`	Define and apply the rule that keeps unfinished tasks (lines 45–56).
`TaskReport`	Requests data through those interfaces, applies the filter, and prints the result (lines 58–79).
`Program.Main`	Creates the sample data and wires the concrete classes to the report (lines 7–19).

The interfaces mark substitution points. They are small because the exercise has one data source and one filter. The code does not add a database, a user interface, or a general-purpose task framework.

2 THREE PRINCIPLES IN THE SOURCE

2.1 Single Responsibility Principle

The three classes split the work:

- `InMemoryTaskSource` supplies the task list (lines 30–43).
- `OpenTaskFilter` selects unfinished tasks (lines 45–56).
- `TaskReport` prints the count and selected rows (lines 58–79).

Storage, selection, and output can change independently.

SRP does not mean one method or one statement per class. `TaskReport` coordinates the source and filter because its single responsibility is to present the filtered task report. The constructor receives those collaborators; their implementation details stay elsewhere.

2.2 Open/Closed Principle

`TaskReport` stores an `ITaskFilter` and calls its `Apply` method (lines 60–72). The current program supplies `OpenTaskFilter`. A future `PriorityTaskFilter` could implement the same interface and select, for example, tasks at priority 2 or higher. That new class would be an extension; `TaskReport` would not need a new branch or a change to its print loop.

The example does not contain `PriorityTaskFilter`; it is a proposed extension, not an implemented or tested feature. `Program.Main` would still need to choose and pass the new filter. The OCP claim here is specifically that the report's filtering behavior can be extended through the interface without editing the report.

2.3 Dependency Inversion Principle

`TaskReport` is the higher-level part of the example: it coordinates producing a filtered report. Its constructor accepts `ITaskSource` and `ITaskFilter`, not `InMemoryTaskSource` or `OpenTaskFilter` (lines 58–67). The concrete choices are made in `Program.Main` (lines 16–18).

This direction lets the report work with another source or filter that honors the same interfaces. For instance, a file-backed source could replace the in-memory source without putting file-reading code inside `TaskReport`. The current project includes only the in-memory implementation; no file-backed source is claimed.

3 TRACE OF THE SAMPLE

The input list is initialized in `Program.Main` (lines 9–14):

ID	Title	Priority	Done?
1	Read the brief	2	No
2	Submit the draft	3	Yes
3	Prepare questions	1	No

The execution path is:

a) `Main` wraps the list in `InMemoryTaskSource` and chooses `OpenTaskFilter` (lines 16–17).

b) It passes both objects to `TaskReport` (line 18).

c) `Print` gets all three tasks from the source and calls `Apply` (line 71).

d) The filter keeps IDs 1 and 3 because their `IsDone` values are false. It removes ID 2 (filter implementation, lines 52–55).

e) The report prints a count of 2 and the remaining rows in their original list order (lines 72–77).

The output implied by that source trace is:

```
Open tasks: 2
1: Read the brief (priority 2)
3: Prepare questions (priority 1)
```

This is a trace-derived example, not a new run measurement. The project README contains the build and run commands.

4 REPRODUCTION AND REVIEW

From the `lab-00` directory, build and run the console project with the installed .NET 10 SDK:

```
& 'D:\apps\dotnet\dotnet.exe' build .\TMPSLab00.csproj
& 'D:\apps\dotnet\dotnet.exe' run --project .\TMPSLab00.csproj
```

The current implementation is intentionally small. It has fixed sample data, one in-memory source, one filter, and console output. It does not demonstrate persistence, changing priorities, or a second filter. Those are possible next extensions, not measured results or existing features.

Questions for oral review:

1. Which separate reasons to change appear in the source, filter, and report classes?
2. How could a priority filter be added while leaving `TaskReport` unchanged? Which composition code would choose it?
3. Why does `TaskReport` receive interfaces instead of constructing its source and filter itself?
4. What would need to change to read tasks from a file? Which existing type could remain unchanged?
5. Which parts of this example are actual code, and which are proposed extensions?

CONCLUSIONS

The simple task list makes three SOLID ideas visible in one source path. SRP separates data access, filtering, and output. OCP provides a filter extension point for `TaskReport`; a new filter is not included in the current code. DIP keeps the report dependent on interfaces and leaves concrete wiring to `Main`. The trace explains why the example prints tasks 1 and 3.

REFERENCES

1. Technical University of Moldova, ELSE. *FAF.TMPS21.1: Laboratory Work #0 - SOLID*. Brief: “Implement 3 SOLID letters in a simple project.” Available online: `https://else.fcim.utm.md/mod/assign/view.php?id=38256` (accessed 10 October 2026).

2. Technical University of Moldova, ELSE. *FAF.TMPS21.1: Tehnici și mecanisme de proiectare software*, section “SOLID Principles.” Available online: `https://else.fcim.utm.md/course/view.php?id=339` (accessed 9 October 2026).

3. R. C. Martin. “Design Principles and Design Patterns,” 2000. Listed in the course references section.

4. Faculty of Computers, Informatics, and Microelectronics, Technical University of Moldova. *Orar secția zi*, academic year 2026/2027. The FAF-241 TMPS seminar lists Daniel Istrati, asist. univ. Available online: `https://fcim.utm.md/orar-sectia-zi-2/` (accessed 9 October 2026).

APPENDICES

APPENDIX A – Program source

The complete source below is the version described in this report. Line numbers in the discussion refer to this listing.

Listing A.1: Task-list project source

```
using System;
using System.Collections.Generic;
using System.Linq;

internal static class Program
{
    private static void Main()
    {
        List<TaskItem> tasks = new()
        {
            new TaskItem(1, "Read the brief", 2, false),
            new TaskItem(2, "Submit the draft", 3, true),
            new TaskItem(3, "Prepare questions", 1, false)
        };

        ITaskSource source = new InMemoryTaskSource(tasks);
        ITaskFilter filter = new OpenTaskFilter();
        TaskReport report = new(source, filter);
        report.Print();
    }
}

internal sealed record TaskItem(int Id, string Title, int Priority, bool IsDone);

internal interface ITaskSource
{
    IReadOnlyList<TaskItem> GetAll();
}

internal sealed class InMemoryTaskSource : ITaskSource
{
    private readonly IReadOnlyList<TaskItem> _tasks;

    public InMemoryTaskSource(IReadOnlyList<TaskItem> tasks)
    {
        _tasks = tasks;
    }

    public IReadOnlyList<TaskItem> GetAll()
```

```
    {
        return _tasks;
    }
}

internal interface ITaskFilter
{
    IEnumerable<TaskItem> Apply(IEnumerable<TaskItem> tasks);
}

internal sealed class OpenTaskFilter : ITaskFilter
{
    public IEnumerable<TaskItem> Apply(IEnumerable<TaskItem> tasks)
    {
        return tasks.Where(task => !task.IsDone);
    }
}

internal sealed class TaskReport
{
    private readonly ITaskSource _source;
    private readonly ITaskFilter _filter;

    public TaskReport(ITaskSource source, ITaskFilter filter)
    {
        _source = source;
        _filter = filter;
    }

    public void Print()
    {
        List<TaskItem> openTasks = _filter.Apply(_source.GetAll()).ToList();
        Console.WriteLine($"Open tasks: {openTasks.Count}");

        foreach (TaskItem task in openTasks)
        {
            Console.WriteLine($"{task.Id}: {task.Title} (priority {task.Priority})");
        }
    }
}
```